

ARKANA



Odoo V18 Technical Training

Prasetiya Mulya University

arkana.co.id



ARKANA

ODOO V19 • TECHNICAL TRAINING

Day 3

Business Logic, State Management, Security & Wizards

Constraints • State Management • Approval Buttons • Security • Wizards

Prasetiya Mulya University
arkana.co.id



Odoo Trainer Technical



Ardian Pramana

Lead Developer @Arkana

Rundown **Training** **Arkana**



- 08.30 - 10.00** : Sesi I
- 10.00 - 10.15** : Coffee Break
- 10.15 - 12.00** : Sesi II
- 12.00 - 13.15** : ISHOMA
- 13.15 - 15.20** : Sesi III
- 15.20 - 15.45** : Break Ashr
- 15.45 - 16.30** : Sesi IV

ARKANA

Follow our **Social Media**



@arkana_academy



@arkana_academy



Arkana Solusi Digital



@arkanasolusidigital

www.arkana.co.id

CONTINUITY

Day 2 Checkpoint

Day 2 selesai — views, relations, dan inheritance:



custom list/form/search views



academy.batch · academy.enrollment · academy.course.tag



relasi M2o / O2m / M2m + header-line



inheritance, computed, default, onchange



Todo

Belum ada aturan (constraint), keamanan (security), atau aksi massal (wizard).

DAY 3 • GOAL

Day 3 Target

Constraints — Python @api.constrains + SQL_sql_constraints di batch & enrollment.



State & Multi-Level Approval — statusbar + approval buttons; submit → L1 → L2 → confirmed; metadata + has_group.



Security for Approval — groups (User/Manager/L1/L2) • access rights • record rules.



Wizard + Monitoring — Reject Enrollment Wizard; approval monitoring views (decoration/search by state).

Inti hari ini: state management & approval



01 Constraints

full hands-on



02 State & Approval

full hands-on



03 Monitoring Views

demo-led



04 Security

full hands-on



05 Wizards

guided



Posisi journey: Day 1 Module & Models → Day 2 Views & Relations → **Day 3 State Management & Approval** → Day 4 Reports & Integrations → Day 5.
Designing Multi-Level Approval dibahas setelah State & Approval.

PART 1

Business Rules with Constraints

Backend menjaga integritas, bukan UI

PART 1 • BUSINESS PROBLEM

***Domain & onchange
bisa dilewati.***

onchange/domain hanya hidup di form UI. Import, RPC, atau kode lain bisa menulis data invalid. Integritas harus dijaga di backend.

UI bisa di-bypass

Import CSV / RPC tak lewat form → onchange & domain tak jalan.

Data bisa korup

capacity 0, end_date < start_date, student terdaftar dobel.

Perlu enforcement

Aturan di level backend untuk semua jalur tulis.

Constraint vs Domain vs Onchange

Tiga lapis berbeda — hanya satu yang melindungi data.



Domain

Filter pilihan di UI/search.

Kapan: saat memilih di form.
Bisa dilewati? Ya (bukan aturan).



Onchange

Bantu isi form (in-memory).

Kapan: saat field berubah di form.
Bisa dilewati? Ya, tak tersimpan.



Constraint

Enforce integritas di backend.

Kapan: create/write (semua jalur).
Bisa dilewati? Tidak.

Python Constraints (@api.constrains)

```
skeleton
from odoo.exceptions import ValidationError

@api.constrains("capacity")
def _check_capacity(self):
    for rec in self:
        if rec.capacity <= 0:
            raise ValidationError(
                "Capacity must be > 0.")
```

@api.constrains(fields)

jalan saat field dipantau berubah.

loop self

recordset bisa banyak — wajib loop.

ValidationError

tolak + pesan jelas ke user.

SQL Constraints (_sql_constraints)

```
model
_sql_constraints = [
    ("unique_student_batch",
     "unique(batch_id, student_id)",
     "A student cannot enroll twice
      in the same batch."),
]
```

DB-level (PostgreSQL)

unique / check — paling kuat & cepat.

atomic

tak bisa di-race oleh dua transaksi.



Perhatian upgrade: jika data lama sudah melanggar (mis. enrollment dobel), pembuatan constraint gagal saat -u. Bersihkan data dulu.

Batch constraints (capacity, tanggal)

```
models/academy_batch.py
@api.constrains("capacity")
def _check_capacity(self):
    for batch in self:
        if batch.capacity <= 0:
            raise ValidationError(
                "Capacity must be > 0.")

@api.constrains("start_date", "end_date")
def _check_dates(self):
    for batch in self:
        if (batch.start_date and batch.end_date
            and batch.start_date > batch.end_date):
            raise ValidationError("start <= end")
```

capacity > 0

tolak kapasitas nol/negatif.

start ≤ end

tanggal mulai tak boleh setelah selesai.

edit file existing

constraints di academy_batch.py.

Enrollment: unique + kapasitas

```
models/academy_enrollment.py
_sql_constraints = [
    ("unique_student_batch",
     "unique(batch_id, student_id)",
     "Student already enrolled.")

@api.constrains("state", "batch_id")
def _check_capacity(self):
    for enr in self:
        if enr.state == "confirmed":
            n = self.search_count([
                ("batch_id", "=", enr.batch_id.id),
                ("state", "=", "confirmed")])
            if n > enr.batch_id.capacity:
                raise ValidationError("Full.")
```

Unique

≈ nomor invoice unik, no double-booking

Kapasitas

≈ qty ≤ stok, seat ≤ kapasitas

PART 1 • LAB CHECKPOINT A

Constraints (full hands-on).

1. Edit academy_batch.py: @api.constrains capacity > 0.
2. Tambah constrains start_date <= end_date.
3. Edit academy_enrollment.py: _sql_constraints unique(batch_id, student_id).
4. Tambah constrains kapasitas confirmed.
5. Upgrade, lalu uji tiap aturan.

**SELESAI BILA**

- capacity 0 ditolak
- tanggal terbalik ditolak
- enroll dobel ditolak (SQL)

```
ing fields for every extended model
2026-06-24 02:39:00,959 13846 ERROR v19_training odoo.schema: could not create unique index "academy_enrollment_check_student_batch"
DETAIL: Key (batch_id, student_id)=(40, 3) is duplicated.
```


Coffee Break

10.00 – 10.15

Waktu Indonesia Barat (WIB)

PART 2

State Management and Approval Buttons

Lifecycle terkontrol + jejak approval berjenjang

PART 2 • BUSINESS PROBLEM

Perubahan status dokumen lewat Approval

Enrollment adalah dokumen yang melewati persetujuan berjenjang. Transisi harus terkontrol (mis. done tak boleh kembali draft) dan butuh jejak: siapa submit, siapa approve L1/L2, kapan.

Transisi valid

hanya status tertentu boleh berpindah ke status berikutnya.

Approval berjenjang

submit → Level 1 → Level 2 → confirmed.

Jejak approval

siapa & kapan menyetujui (audit trail).

State field & statusbar

```
models/academy_enrollment.py + form
state = fields.Selection([
    ("draft", "Draft"),
    ("submitted", "Submitted"),
    ("manager_approved", "Manager Approved"),
    ("confirmed", "Confirmed"),
    ("done", "Done"),
    ("rejected", "Rejected"),
    ("cancelled", "Cancelled")], default="draft")
```

```
<field name="state" widget="statusbar"
    statusbar_visible="draft,submitted,
    manager_approved,confirmed,done"/>
```

Selection

status sebagai daftar nilai tetap.

confirmed = final approval

kompatibel constraint kapasitas Day 3.

statusbar_visible

langkah utama yang tampil di header.

Header button (type=object) + transition validation

```
button method
<button name="action_submit" type="object"
    string="Submit" invisible="state != 'draft'"/>

def action_submit(self):
    for rec in self:
        if rec.state != "draft":
            raise UserError(
                "Only draft can be submitted.")
        rec.write({"state": "submitted",
            "submitted_by_id": self.env.user.id,
            "submitted_date": fields.Datetime.now() })
```

type=object

button memanggil method di model.

loop self

aksi bisa multi-record — wajib loop.

validasi transisi

cek state + UserError sebelum write.

Approval metadata (audit trail)

```
models/academy_enrollment.py
submitted_by_id = fields.Many2one(
    "res.users", readonly=True)
submitted_date = fields.Datetime(readonly=True)
manager_approved_by_id = fields.Many2one(
    "res.users", readonly=True)
manager_approved_date = fields.Datetime(...)
final_approved_by_id = fields.Many2one(...)
final_approved_date = fields.Datetime(...)
rejection_reason = fields.Text(readonly=True)
```

readonly

diisi otomatis oleh method, bukan manual.

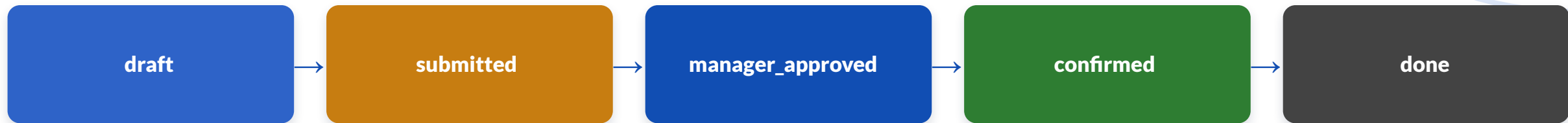
per level

submitted / L1 / final tercatat terpisah.

audit dasar

siapa & kapan; bisa diperkaya chatter/activity.

Multi-level approval flow



Branching: Submitted / Manager Approved → Reject → Rejected • Rejected / Cancelled → Reset → Draft

Sequential

submit → L1 (manager_approved) → L2 (confirmed) → done; tiap langkah satu method + satu group.

Reject & Reset

reject dari submitted/manager_approved; reset dari rejected/cancelled → draft.

PART 2 • ODOO CONCEPT — MENTAL MODEL

Button visibility is not security.



UI controls

```
invisible="state != ..."
```

```
groups="...approval_11"
```

Hanya menyembunyikan tombol. User tetap bisa memanggil method via RPC / script.



Backend security

access rights + record rules

validasi transisi di Python

```
self.env.user.has_group(  
    "...approval_11/_12")
```


Enrollment approval berjenjang

```
models/academy_enrollment.py
def action_manager_approve(self):
    if not self.env.user.has_group(
        "academy_management.\
academy_group_approval_l1"):
        raise UserError("Not allowed (L1).")
    for rec in self:
        if rec.state != "submitted":
            raise UserError("Only submitted.")
        rec.write({"state": "manager_approved",
            "manager_approved_by_id": self.env.user.id,
            "manager_approved_date":
                fields.Datetime.now() })
```

Flow demo

draft → Submit → (login L1) Level 1 Approve →
(login L2) Final Approve → Confirmed → Done.
User biasa panggil method → UserError.

ERP • Approval berjenjang

≈ PO approval • expense approval • leave approval

PART 2 • LAB CHECKPOINT B

State Management and Multi-Level Approval.

1. Edit academy_enrollment.py: state 7-status + approval metadata.
2. Tulis 5 approval method (submit/L1/L2/done/reset) + helper _reject_with_reason + has_group L1/L2.
3. Edit academy_groups.xml: group Approval L1 & L2; manager implies keduanya.
4. Edit enrollment form: header buttons + statusbar + Approval Info page.
5. Upgrade, assign 3 test user (L1/L2/biasa), uji lifecycle.

**SELESAI BILA**

- transisi berjenjang jalan
- L1/L2 dibatasi backend (has_group)
- user biasa ditolak via RPC



40 menit • Detail lengkap & full code di Lab Pack — Checkpoint B.

PART 3

Designing Multi-Level Approval in Odoo

Possibility & effort — bukan sekadar syntax

Pola desain approval di Odoo



Simple / Fixed

state + button + group; atau state machine + approval groups per level (seperti lab ini).

Low-Medium.



Approval Matrix

approver ditentukan data: amount / department / company (tabel konfigurasi).

Medium-High.



Generic Engine

satu mekanisme approval lintas dokumen via mixin/adapter; konfigurasi terpusat.

High.



Dimensi lain: sequential vs parallel • reject/return/reset • delegation & escalation (future scope) • audit via chatter/activity.

Kebutuhan → pattern → effort

Kebutuhan	Pattern	Effort
1 approval sederhana	state + button + group	Low
2–3 level fixed approval	state machine + approval groups	Medium
approval by amount/department/company	approval matrix	Medium–High
approval hampir semua dokumen	shared approval engine + adapters	High
SLA, escalation, delegation, parallel	workflow engine extension	High

Lab hari ini = baris ke-2 (Medium): state machine + approval groups L1/L2 pada satu dokumen. Bukti possibility + titik acuan effort.

8 pertanyaan penentu desain

- 1 Approval fixed atau by amount / department / company / branch?
- 2 Sequential atau parallel?
- 3 Approver: user, group, role, position, atau supervisor?
- 4 Saat reject: ke draft, ke approver sebelumnya, atau cancelled?
- 5 Perlu delegation (approver berhalangan)?
- 6 Perlu escalation / SLA (deadline approval)?
- 7 Generic lintas dokumen atau document-specific?
- 8 Dokumen apa: purchase, sales, invoice, expense, inventory, custom?



Odoo sangat memungkinkan approval berjenjang. Yang dianalisis bukan apakah bisa, tetapi desain mana yang tepat — simple workflow, approval matrix, atau generic engine. Effort bergantung kompleksitas aturan.

ISHOMA

11.50 – 13.05

Waktu Indonesia Barat (WIB)

PART 4

Advanced Views for Approval Monitoring

Memantau status approval (demo-led)

Monitoring approval: decoration & search by state

```
views/academy_enrollment_views.xml
<list decoration-info="state=='draft'"
  decoration-warning="state=='submitted'"
  decoration-primary="state=='manager_approved'"
  decoration-success="state=='confirmed'"
  decoration-danger="state=='rejected'">

<search>
  <filter name="submitted" string="Submitted"
    domain="[('state','=', 'submitted')]" />
  <filter name="grp_state" context=
    "{ 'group_by': 'state' }" />
</search>
```

decoration by state

warna baris per status approval.

search filter by state

Submitted / Manager Approved / Confirmed / Rejected.

kanban by state (optional)

papan status approval bila sempat.

Approval monitoring

TRAINER DEMO



List decoration by state baris berwarna sesuai status approval.



Search filter by approval state filter Submitted / Manager Approved / Confirmed / Rejected.



Kanban by state (opsional) papan status approval; drag bila relevan.



Calendar & Graph awareness saja — untuk jadwal & tren, bukan fokus hari ini.

PART 4 • LAB CHECKPOINT C (OPTIONAL)

Approval monitoring – mini task.

1. Mulai dari source checkpoint (monitoring views sudah ada).
2. Opsional pilih SATU:
3.
 - tambah 1 search filter by approval state, atau
4.
 - tambah 1 list decoration by state.
5. Upgrade & verifikasi tampilan.



~15 menit (opsional) • Versi lengkap tersedia di source checkpoint.

**SELESAI BILA**

- paham decoration/filter by state
- bisa baca arch search
- 1 perubahan kecil tampil

PART 5

Security for Approval

Groups, access rights, record rules — termasuk approver L1/L2

PART 5 • BUSINESS PROBLEM

Melihat menu bukan berarti boleh.

Semua user kini melihat semua batch & enrollment. Trainer hanya kelola batch miliknya; manager boleh semua. Selain itu: siapa boleh approve L1/L2? Butuh kontrol per-model, per-record, dan per-aksi.

Akses model

boleh baca/tulis model apa? → access rights.

Akses record

record mana yang boleh dilihat/approve? → record rules.

Izin aksi bisnis

siapa boleh approve L1/L2? → group + has_group (bukan sekadar tombol).

PART 5 • ODOO CONCEPT — MENTAL MODEL

Tiga pertanyaan keamanan yang berbeda.



Access rights

“Boleh akses MODEL ini?”

read • write • create • unlink.
Model-level. ir.model.access.csv per
group.



Record rules

“Record MANA yang boleh?”

Record-level via domain. ir.rule
domain_force, mis. responsible_id
== user.



Domain

Filter UI / search.

BUKAN security. Hanya
kenyamanan tampilan; data tetap di
backend.

PART 5 • ODOO CONCEPT — ROLE MODEL

Base Access vs Approval Authority vs Administration



Base Access

Academy User

Akses dasar ke menu, model, dan record. Titik awal tiap persona.



Approval Authority

Approver L1 / L2

Izin aksi approval tertentu (L1/L2). Group = permission set, bukan selalu jabatan.



Administration

Manager / Administrator

Full access, konfigurasi, recovery. Manager implies User + L1 + L2. Bukan Admin pun bisa jadi Approver.

Groups (res.groups) + approval levels

```
security/academy_groups.xml
<record id="academy_group_approval_l1"
        model="res.groups">
  <field name="name">Academy Approval Level 1</field>
  <field name="implied_ids"
        eval="[(4, ref('academy_group_user'))]" />
</record>
<!-- Manager implies User + L1 + L2 -->
<field name="implied_ids" eval="[(4, 11), (4, 12)]" />
```

User / Manager

akses dasar; Manager implies semua.

Approval L1 / L2

izin approve bertingkat (implies User).

group ≠ keamanan tombol

approval tetap dicek has_group() di method.

Access Rights (ir.model.access.csv)

Hak CRUD per (model, group). Pisahkan User vs Manager. Tanpa baris = tak ada akses untuk group itu.

model	group	read	write	create	unlink
academy.course	Academy User	1	0	0	0
academy.course	Academy Manager	1	1	1	1
academy.enrollment	Academy User	1	1	1	0
academy.enrollment	Academy Manager	1	1	1	1

User: course read-only, enrollment read/write/create. Manager: full CRUD. Model-level – bukan record-level.

Record Rules (ir.rule)

```
security/academy_record_rules.xml
<record id="batch_rule_user_own"
        model="ir.rule">
  <field name="model_id"
        ref="model_academy_batch"/>
  <field name="groups"
        eval="[(4, ref('academy_group_user'))]" />
  <field name="domain_force">
    [('responsible_id','=',user.id)]</field>
</record>
```

domain_force

filter record per group.

user.id

konteks user saat ini.

global vs group

tanpa group = berlaku semua (hati-hati).

Tiga file security standar

```
ir.model.access.csv  
id,name,  
model_id:id,  
group_id:id,  
perm_read,...  
  
access_x,...,  
  group_user,  
  1,0,0,0
```

```
res.groups (XML)  
<record  
  model=  
    "res.groups">  
  name,  
  category_id,  
  implied_ids  
</record>
```

```
ir.rule (XML)  
<record  
  model=  
    "ir.rule">  
  model_id,  
  groups,  
  domain_force  
</record>
```

Urutan load di manifest: groups → access csv → record rules. Pola sama di semua module Odoo.

User vs Manager + record rules (batch & enrollment)

```
academy_record_rules.xml
<!-- BATCH: user lihat miliknya -->
<field name="domain_force">
    [('responsible_id','=',user.id)]</field>

<!-- ENROLLMENT: ikut jalur akses -->
<field name="domain_force">
    [('batch_id.responsible_id','=',
    user.id)]</field>

<!-- MANAGER (batch & enrollment) -->
<field name="domain_force">
    [(1,'=',1)]</field>
```



Kebocoran data

Batasi batch saja → user masih lihat enrollment batch orang lain lewat Enrollment list. Record rule harus ikut JALUR AKSES DATA.

ERP MAPPING salesperson lihat dokumen sendiri, manager semua

responsible_id default = user saat ini. Manager: implies User + rule "all".

PART 5 • LAB CHECKPOINT D

Security (full hands-on).

1. buat groups (User & Manager, implies). 2. access rights per group. 3. tambah responsible_id. 4. record rule batch & enrollment. 5. uji dengan 2 user.



SECURITY RECOVERY – WAJIB

- Jangan uji record rule pakai admin (admin bypass).
- Buat 2 test user: Academy User & Academy Manager.
- Simpan admin/superuser untuk recovery bila terkunci.
- Record rule terlalu ketat → user tak lihat data sama sekali.
- Domain bukan security. Verifikasi Manager lihat semua.
- Jangan buat global rule tanpa paham efeknya.

Selesai bila: User terbatas (batch & enrollment), Manager penuh, tanpa kebocoran. 🕒 40 menit • Lihat Lab Pack D.

PART 6

Wizards for Approval Actions

Aksi terpandu pada record terpilih

PART 6 • BUSINESS PROBLEM

Beberapa aksi butuh dialog, bukan record permanen.

Menolak enrollment butuh alasan tertulis dan hanya boleh dari state submitted / manager_approved — lewat dialog sementara, bukan model permanen.

Mass action

reject record terpilih sekaligus.

Mengumpulkan input

alasan penolakan sebelum aksi.

Sementara

data dialog tak perlu disimpan permanen.

TransientModel

```
wizard model
class RejectEnrollmentWizard(
    models.TransientModel):
    _name = "academy.enrollment.\
        reject.wizard"
    rejection_reason = fields.Text(
        string="Rejection Reason",
        required=True)
```

TransientModel

record sementara (auto-vacuum).

seperti Model

punya fields & methods.

butuh access rights

tetap perlu ir.model.access.

Wizard form • action target=new • context

```
wizard view + action
<form>
  <group>
    <field name="rejection_reason"/></group>
  <footer>
    <button name="action_reject"
      type="object" string="Reject"/>
  </footer>
</form>
<!-- action -->
<field name="target">new</field>
<field name="binding_model_id"
  ref="model_academy_enrollment"/>
```

target=new

buka form sebagai dialog.

binding_model_id

muncul di menu Action list.

context.active_ids

berisi record terpilih.

Pola wizard standar (disederhanakan)

```
pola umum
def action_reject(self):
    ids = self.env.context.get(
        "active_ids", [])
    records = self.env["academy.\
        enrollment"].browse(ids)
    records.write({...})
    return {"type":
        "ir.actions.act_window_close"}
```

active_ids dari context
record terpilih di list.

browse(ids)
jadikan recordset.

aksi + close
write lalu tutup dialog.

Reject Enrollment Wizard

```
wizards/reject_enrollment_wizard.py
def action_reject(self):
    ids = self.env.context.get("active_ids", [])
    enrollments = self.env[
        "academy.enrollment"].browse(ids)
    enrollments._reject_with_reason(
        self.rejection_reason)
    return {
        "type": "ir.actions.act_window_close"}
```

TRAINER DEMO

Flow demo

pilih submitted/manager_approved → Action → Reject Enrollment → isi alasan → Reject → state = rejected

ERP • Aksi massal

≈ reject/return document • cancel with reason • approval rejection



Selaras approval: Logic reject tetap terpusat di model melalui `_reject_with_reason()`. Wizard hanya mengumpulkan alasan dan memanggil helper.

Ashar Break

15.10 – 15.45

Waktu Indonesia Barat (WIB)

PART 6 • LAB CHECKPOINT E

Wizard (guided hands-on).

1. create/verify wizard model (TransientModel).
2. create/verify wizard view + action (target=new + binding).
3. add/verify wizard access rights (ir.model.access).
4. test: pilih enrollment (submitted) → Action → Reject Enrollment.
5. verify: state berubah rejected (rejection_reason terisi).

**SELESAI BILA**

- wizard terbuka dari list
- active_ids terbaca
- reject dengan alasan jalan



25-30 menit • Guided — pakai source checkpoint E sebagai reference (bukan from-scratch).

Jebakan tersering Day 3

**button visibility dikira security**

groups/invisible = UI; backend tetap cek has_group().

**approval method tanpa has_group**

L1/L2 wajib has_group() di Python, bukan hanya tombol.

**domain sebagai security**

domain = filter UI, bukan keamanan.

**record rule terlalu ketat**

domain salah → user tak lihat apa pun.

**SQL constraint gagal upgrade**

data dobel lama → bersihkan dulu.

**transisi state tak divalidasi**

selalu cek state + UserError sebelum write.

**constraint tak loop self**

selalu for rec in self.

**lupa access rights wizard**

TransientModel butuh ir.model.access.

**L1/L2 tak implies User**

approver tak punya akses baca/tulis enrollment.

**Model untuk wizard**

harus models.TransientModel; baca active_ids.

Aplikasi kini terkontrol, aman, dan interaktif



Constraints

Python + SQL — integritas data.



State & Approval

statusbar + multi-level buttons +
has_group.



Security

groups (User/Manager/L1/L2) + record
rules.



Wizard + Monitoring

TransientModel; view monitoring by
state.



Deliverable Day 3 tercapai: academy_management kini punya constraints, state management & approval berjenjang (submit → L1 → L2 → confirmed), security (User/Manager/L1/L2), monitoring views, dan Reject Enrollment Wizard.

Data sudah terkontrol & aman. Day 4: keluarkan data (report) & hubungkan sistem lain (API).

NEXT • DAY 4

Reports & Integrations.



PDF reports (QWeb) — dokumen & sertifikat Academy.



Excel reports — export data terstruktur.



Report inheritance — extend report standar.



REST / XML-RPC / JSON-RPC — consumer & integrasi sistem lain.

WRAP-UP

Review, QnA & Upcoming Preview

Pertanyaan, blocker lab, dan cuplikan Day 4

Terima Kasih

Prasetiya Mulya University · Odoo V18 Technical Training

ARKANA

www.arkana.co.id